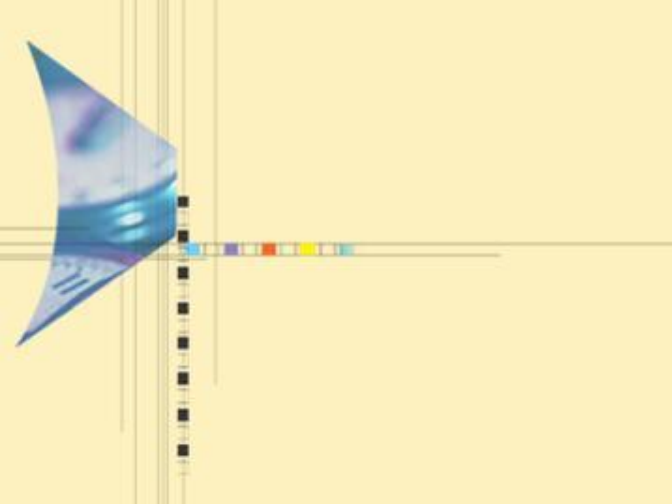




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）

主要内容



1.1 算法的基本概念

1.2 算法分析基础

1.3 算法时间估算方法

1.4 求解递推关系



主要内容



1.4 求解递推关系

- 递推关系求解方法
- 迭代法
- 尝试法
- 主定理



1.4.1 递推关系的求解

递归公式是用它自身来定义的一个公式，在这种情况下，我们称这个定义为**递推关系或递推式**。

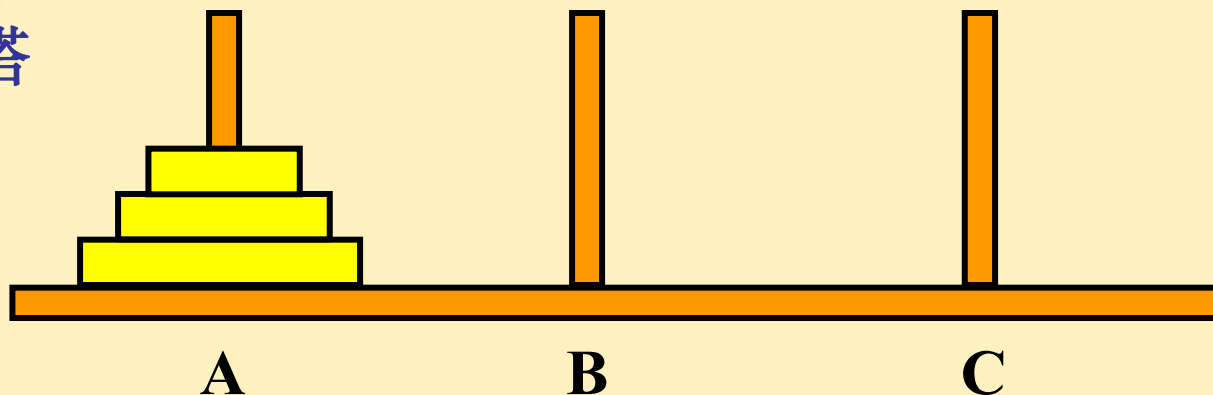
递推式的一般求解方法：

- 迭代法
 - 直接迭代：插入排序最坏情况下时间分析
 - 换元迭代：二分归并排序最坏情况下时间分析
 - 差消迭代：快速排序平均情况下的时间分析
 - 迭代模型：递归树
- 尝试法：快速排序平均情况下的时间分析
- 主定理：递归算法的分析



1.4.2 迭代法求解

例: Hanoi塔



$\text{Hanoi}(A, C, n)$ // 将A的 n 个盘子按要求移到C
1. if $n=1$ then move (A, C) // 将A的1个盘子移到C
2. else $\text{Hanoi}(A, B, n-1)$
3. move (A, C)
4. $\text{Hanoi}(B, C, n-1)$



1.4.2 迭代法求解

例：Hanoi塔

$$T(n) = 2 T(n-1) + 1, \quad T(1) = 1,$$

迭代求解

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2[2T(n-2) + 1] + 1 = 2^2T(n-2) + 2 + 1 \\ &= 2^3T(n-3) + 2^2 + 2 + 1 = \dots \\ &= 2^{n-1}T(1) + 2^{n-2} + \dots + 2 + 1 \\ &= 2^n - 1 \end{aligned}$$

1秒移1个，64个盘子要多少时间？(5000亿年)

1.直接迭代法-插入排序算法最坏情况下时间分析

输入大小为 n ，在最坏情况下，元素比较操作执行的总次数为 $W(n)$ ，则

$$\begin{cases} W(n) = W(n-1) + n - 1 \\ W(1) = 0 \end{cases}$$

$$\begin{aligned} W(n) &= W(n-1) + n - 1 \\ &= [W(n-2) + n - 2] + n - 1 = W(n-2) + (n-2) + (n-1) \\ &= [W(n-3) + n - 3] + (n-2) + (n-1) = \dots \\ &= W(1) + 1 + 2 + \dots + (n-2) + (n-1) \\ &= 1 + 2 + \dots + (n-2) + (n-1) = n(n-1)/2 \end{aligned}$$

2.换元迭代法-归并排序算法最坏情况下时间分析

MergeSort(A, p, r) // 归并排序数组 $A[p..r]$

1. if $p < r$
2. then $q \leftarrow \lfloor (p+r)/2 \rfloor$
3. **MergeSort(A, p, q)**
4. **MergeSort($A, q+1, r$)**
5. **Merge(A, p, q, r)**



2.换元迭代法-归并排序算法最坏情况下时间分析

输入大小为 n ，在最坏情况下，元素比较操作执行的总次数为 $W(n)$ ，则

$$\begin{cases} W(n) = 2W(n/2) + n - 1 \\ W(1) = 0 \end{cases}$$

令 $n=2^k$ ，代入得

$$\begin{aligned} W(n) &= 2W(2^{k-1}) + 2^k - 1 \\ &= 2[2W(2^{k-2}) + 2^{k-1} - 1] + 2^k - 1 \\ &= 2^2W(2^{k-2}) + 2^k - 2 + 2^k - 1 \\ &= 2^2[2W(2^{k-3}) + 2^{k-2} - 1] + 2^k - 2 + 2^k - 1 = \dots \\ &= 2^k W(1) + k2^k - (2^{k-1} + 2^{k-2} + \dots + 2 + 1) \\ &= k2^k - 2^k + 1 \\ &= n \log n - n + 1 \end{aligned}$$

3.差消迭代法-快速排序算法平均情况下时间分析

quicksort(A,low,high)

1. **if** $low < high$ **then**
2. *SPLIT*($A[low..high], w$) { w is the new position of $A[low]$ }
3. *quicksort*($A, low, w-1$)
4. *quicksort*($A, w+1, high$)
5. **end if**

3.差消迭代法-快速排序算法平均情况下时间分析

输入大小为 n ，在平均情况下，元素比较操作执行的总次数为 $T(n)$ ，则

$$\begin{cases} T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + cn, & n \geq 2 \\ T(1) = 0 \end{cases}$$

$$nT(n) = 2 \sum_{i=1}^{n-1} T(i) + cn^2$$

$$(n-1)T(n-1) = 2 \sum_{i=1}^{n-2} T(i) + c(n-1)^2$$

$$nT(n) = (n+1)T(n-1) + 2cn - c$$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{2cn-c}{n(n+1)} = \dots$$

$$= 2c \left[\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3} + \frac{T(1)}{2} \right] - O\left(\frac{1}{n}\right) = \Theta(\log n)$$

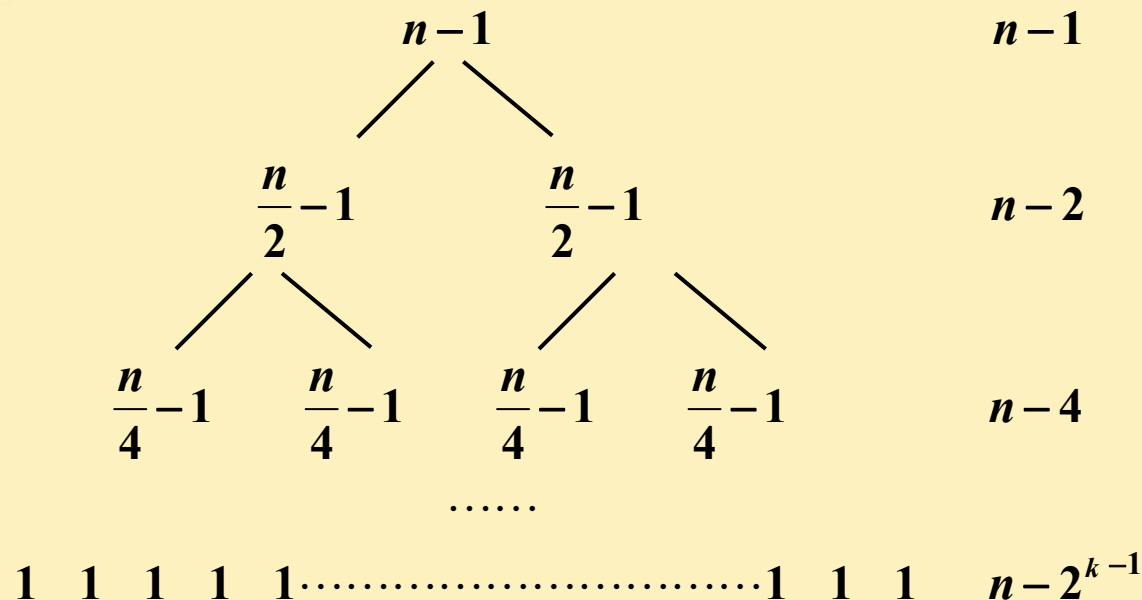
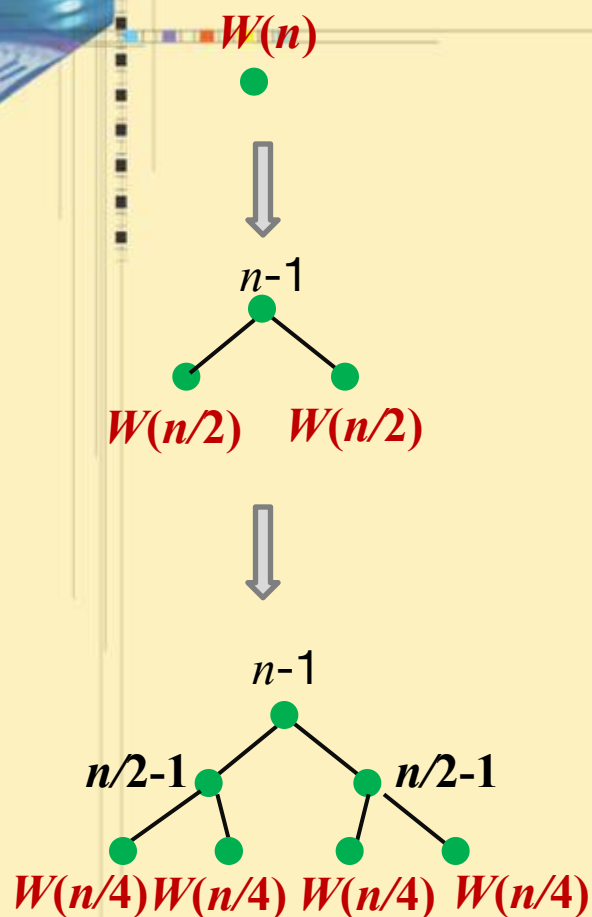
$$T(n) = \Theta(n \log n)$$

4.迭代模型-递归树

- 递归树是一棵结点带权的二叉树，初始的递归树只有一个结点，权标记为函数自身。然后不断迭代，直到树中不再含有权为函数的结点为止。
- 迭代规则就是将递归树中权为函数的结点用和这个函数相等的递推方程右边的子树代替。子树只有2层，根标记为方程右部除了函数外的剩余表达式，每一片叶子则代表函数右部的一个递归函数项。
- 一直迭代下去，直到树叶不再有函数为止。整个递归树中全部结点的权和不不变，总等于函数本身。
- 为了计算最终的递归树中所有结点的权之和，可以采用分层计算的方法。
- 例如，假设递归函数如下定义：

$$\begin{cases} W(n) = 2W(n/2) + n - 1 & n=2^k \\ W(1) = 0 \end{cases}$$

生成过程



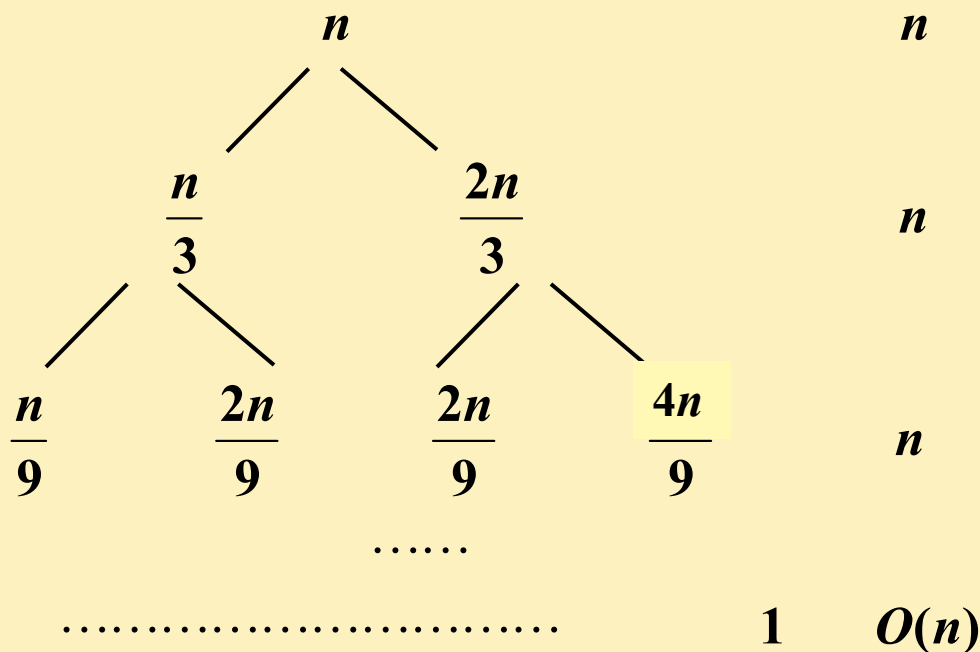
$$W(n) = 2W(n/2) + n-1, \quad n=2^k, \quad W(2)=1.$$

$$W(n) = n-1 + n-2 + \dots + n-2^{k-1}$$

$$= kn - (2^k-1) = n \log n - n + 1$$

递归树的应用实例

求解: $T(n) = T(n/3) + T(2n/3) + n$



层数 k : $n(2/3)^k = 1 \Rightarrow (3/2)^k = n \Rightarrow k = O(\log_{3/2} n)$
 $T(n) = O(n \log n)$

1.4.3 尝试法

估计递推方程的阶，也可以使用尝试法。

基本思想：

首先猜想解是哪种类型的函数，给出这个函数的一般表达形式，表达式中可能含有某些待定参数。然后将这个函数代入原递推方程以确定参数的值



尝试法-快速排序平均情况分析

(1) $T(n)=C$ 为常函数, 左边= $O(1)$

$$\text{右边} = \frac{2}{n} C(n-1) + O(n) = 2C - \frac{2C}{n} + O(n)$$

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + O(n)$$

(2) $T(n)=cn$, 左边= cn

$$\text{右边} = \frac{2}{n} \sum_{i=1}^{n-1} ci + O(n) = \frac{2c}{n} \frac{(1+n-1)(n-1)}{2} + O(n) = cn - c + O(n)$$

(3) $T(n)=cn^2$, 左边= cn^2

$$\text{右边} = \frac{2}{n} \sum_{i=1}^{n-1} ci^2 + O(n) = \frac{2}{n} \left[\frac{cn^3}{3} + O(n^2) \right] + O(n) = \frac{2c}{3} n^2 + O(n)$$

(4) $T(n)=cn \log n$, 左边= $cn \log n$

$$\text{右边} = \frac{2c}{n} \sum_{i=1}^{n-1} i \log i + O(n) = cn \log n + O(n)$$

因此, $T(n) = \Theta(n \log n)$

1.4.4 主定理

定理 设 $a \geq 1$, $b > 1$ 为常数, $f(n)$ 为函数, $T(n)$ 为非负整数, 且

$$T(n) = aT(n/b) + f(n)$$

则有以下结果:

1. 若 $f(n) = O(n^{\log_b a - \varepsilon})$, $\varepsilon > 0$, 那么 $T(n) = \Theta(n^{\log_b a})$
2. 若 $f(n) = \Theta(n^{\log_b a})$, 那么 $T(n) = \Theta(n^{\log_b a} \log n)$
3. 若 $f(n) = \Omega(n^{\log_b a + \varepsilon})$, $\varepsilon > 0$, 且对某个常数 $c < 1$ 和所有充分大的 n 有 $a f(n/b) \leq c f(n)$, 那么
$$T(n) = \Theta(f(n))$$



主定理的证明

不妨设 $n=b^k$

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

$$= a\left[aT\left(\frac{n}{b^2}\right) + f\left(\frac{n}{b}\right)\right] + f(n) = a^2T\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n)$$

= ...

$$= a^k T\left(\frac{n}{b^k}\right) + a^{k-1} f\left(\frac{n}{b^{k-1}}\right) + \dots + af\left(\frac{n}{b}\right) + f(n)$$

$$= a^k T(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \quad T(1) = c_1$$



情况1

$$f(n) = O(n^{\log_b a - \varepsilon})$$

$$T(n) = c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$= c_1 n^{\log_b a} + O\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a - \varepsilon}\right)$$

$$= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{(b^{\log_b a - \varepsilon})^j}\right)$$

$$= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \sum_{j=0}^{\log_b n - 1} (b^\varepsilon)^j\right)$$

$$= c_1 n^{\log_b a} + O\left(n^{\log_b a - \varepsilon} \frac{b^{\varepsilon \log_b n} - 1}{b^\varepsilon - 1}\right)$$

$$= c_1 n^{\log_b a} + O(n^{\log_b a - \varepsilon} n^\varepsilon) = \Theta(n^{\log_b a})$$



情况2

$$f(n) = \Theta(n^{\log_b a})$$

$$T(n) = c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$= c_1 n^{\log_b a} + \Theta\left(\sum_{j=0}^{\log_b n - 1} a^j \left(\frac{n}{b^j}\right)^{\log_b a}\right)$$

$$= c_1 n^{\log_b a} + \Theta\left(n^{\log_b a} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{a^j}\right)$$

$$= c_1 n^{\log_b a} + \Theta(n^{\log_b a} \log n)$$

$$= \Theta(n^{\log_b a} \log n)$$



情况3

$$f(n) = \Omega(n^{\log_b a + \varepsilon})$$

$$T(n) = c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right)$$

$$\leq c_1 n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} c^j f(n) \quad \left(af\left(\frac{n}{b}\right) \leq cf(n)\right)$$

$$= c_1 n^{\log_b a} + f(n) \frac{c^{\log_b n} - 1}{c - 1}$$

$$= c_1 n^{\log_b a} + \Theta(f(n)) \quad (c < 1)$$

$$= \Theta(f(n)) \quad (f(n) = \Omega(n^{\log_b a + \varepsilon}))$$



主定理的应用

例1. 求解递推方程 $T(n) = 9T(n/3) + n$

解 上述递推方程中的 $a = 9, b = 3, f(n) = n$, 那么

$$n^{\log_3 9} = n^2, \quad f(n) = O(n^{\log_3 9 - 1}),$$

相当于主定理的第一种情况, 其中 $\varepsilon = 1$. 根据定理得到

$$T(n) = \Theta(n^2)$$

例2. 求解递推方程 $T(n) = T(2n/3) + 1$

解 上述递推方程中的 $a = 1, b = 3/2, f(n) = 1$, 那么

$$n^{\log_{3/2} 1} = n^0 = 1, \quad f(n) = 1$$

相当于主定理的第二种情况. 根据定理得到.

$$T(n) = \Theta(n^0 \log n) = \Theta(\log n)$$

主定理的应用

例3. 求解递推方程

$$T(n) = 3T(n/4) + n \log n$$

解 上述递推方程中的 $a=3, b=4, f(n)=n \log n$, 那么

$$n \log n = \Omega(n^{\log_4 3 + \varepsilon}) = \Omega(n^{0.793 + \varepsilon}), \quad \varepsilon \approx 0.2$$

此外, 要使 $a f(n/b) \leq c f(n)$ 成立, 代入 $f(n)=n \log n$, 得到

$$\frac{3n}{4} \log \frac{n}{4} \leq c n \log n$$

显然只要 $c \geq 3/4$, 上述不等式就可以对充分大的 n 成立.
相当于主定理的第三种情况. 因此有

$$T(n) = \Theta(f(n)) = \Theta(n \log n)$$

不能使用主定理的例子

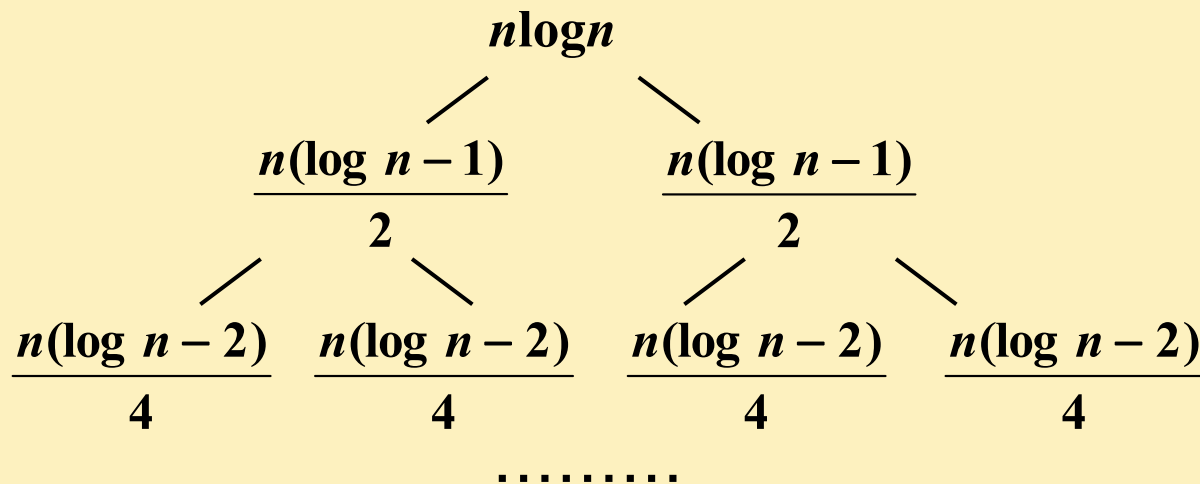
例4. 求解 $T(n) = 2T(n/2) + n \log n$

$$a=b=2, n^{\log_2 2} = n, f(n)=n \log n$$

不存在 $\varepsilon > 0$ 使得 $n \log n = \Omega(n^{1+\varepsilon})$ 成立,



递归树求解 $T(n) = 2T(n/2) + n \log n$



$n \log n$

$n(\log n - 1)$

$n(\log n - 2)$

$$\begin{aligned} T(n) &= n \log n + n(\log n - 1) + n(\log n - 2) + \dots + n(\log n - k + 1) \\ &= (n \log n) \log n - n(1 + 2 + \dots + k - 1) \\ &= n \log^2 n - nk(k - 1)/2 = O(n \log^2 n) \end{aligned}$$